FreyrSCADA

Embedded Solution

ARM GCC Compiler for Beagle bone and raspberry setup:

```
sudo apt-get install build-essential git
```

```
mkdir rpi
```

```
cd rpi
```

```
git clone git://github.com/raspberrypi/tools.git
```

```
cd ~/rpi/tools/arm-bcm2708
```

- In the arm-bcm2708 folder you'll see four other folders, each containing a separate toolchain:

1. arm-bcm2708-linux-gnueabi
2. arm-bcm2708hardfp-linux-gnueabi
3. gcc-linaro-arm-linux-gnueabihf-raspbian
4. gcc-linaro-arm-linux-gnueabihf-raspbian-x64

If you are running a 32-bit Linux operating system, you'll want to use the third toolchain 'gcc-linaro-arm-linux-gnueabihf-raspbian'. If however you are running a 64-bit Linux operating system, you'll want to use the fourth toolchain 'gcc-linaro-arm-linux-gnueabihf-raspbian-x64. If you don't know whether you are running 32-bit or a 64-bit version of Linux type in the command line :

```
uname -a
```

If you see something like 'i386'/'386' in the output string, then you're running a 32-bit version of Linux. If however you see 'x86_64' / 'amd64' in the output string, then you're running a 64-bit version of Linux.

The next step is to add the directory containing the binary files of the toolchain to the PATH environment variable in Linux. This way we can access the toolchain's binary files/tools from the command line. This step is not necessary if you want to using the toolchain strictly from Eclipse. I still do highly recommend that you do it, in case you decide to compile/debug arm applications directly from the command line later on. We will do this by adding an "export PATH" command to the bottom of the .bashrc files in the home directory.

- In a terminal window (on your Desktop Linux OS) type:

```
cd ~/
```

to point to the home directory, then type:

```
nano .bashrc
```

This will open the .bashrc file in a command line based editor called nano. Go to the bottom of the .bashrc file using the down arrow key. Then if you're running 32-bit Linux, type the following command:

```
export PATH=$PATH:$HOME/rpi/tools/arm-bcm2708/gcc-linaro-arm-linux-gnueabihf-raspbian/bin
```

Alternatively if you're running 64-bit Linux, type the following command instead:

```
export PATH=$PATH:$HOME/rpi/tools/arm-bcm2708/gcc-linaro-arm-linux-gnueabihf-raspbian-x64/bin
```

- Then hit Ctrl+x to exit. You will be prompted to save changes. Say yes and hit "Enter".
- Then type in the command line:

```
source .bashrc
```

This allows the current open terminal to see the updated PATH variable. Alternatively one can close the terminal and open a new one.

- To test if you can access the cross compiling toolchain from the command line type:

```
arm-linux-gnueabihf-gcc -v
```

If you were successful, you should see output similar to that in Figure 1. Congratulations! you just installed Raspbian's official cross compiling toolchain on your Desktop PC / Virtual machine!

Compiler and debugger settings

Global compiler settings

Batch builds

Debugger settings

Selected compiler

rasperry arm GNU GCC Compiler

Set as default | Copy | Rename | Delete | Reset defaults

Compiler settings | Linker settings | Search directories | Toolchain executables

Compiler's installation directory

:hain/tools/arm-bcm2708/gcc-linaro-arm-linux-gnueabihf-raspbian/bin .. Auto-detect

NOTE: All programs below, must exist either in the "bin" sub-directory of this path or in any of the "Additional paths"...

Program Files | Additional Paths

C compiler:	arm-linux-gnueabihf-gcc
C++ compiler:	arm-linux-gnueabihf-g++
Linker for dynamic libs:	arm-linux-gnueabihf-g++
Linker for static libs:	arm-linux-gnueabihf-ar
Debugger:	arm-linux-gnueabihf-gdb
Resource compiler:	
Make program:	make

Cancel | OK

Compiler and debugger settings

Global compiler settings

Batch builds

Debugger settings

Selected compiler

rasperry arm GNU GCC Compiler

Set as default | Copy | Rename | Delete | Reset defaults

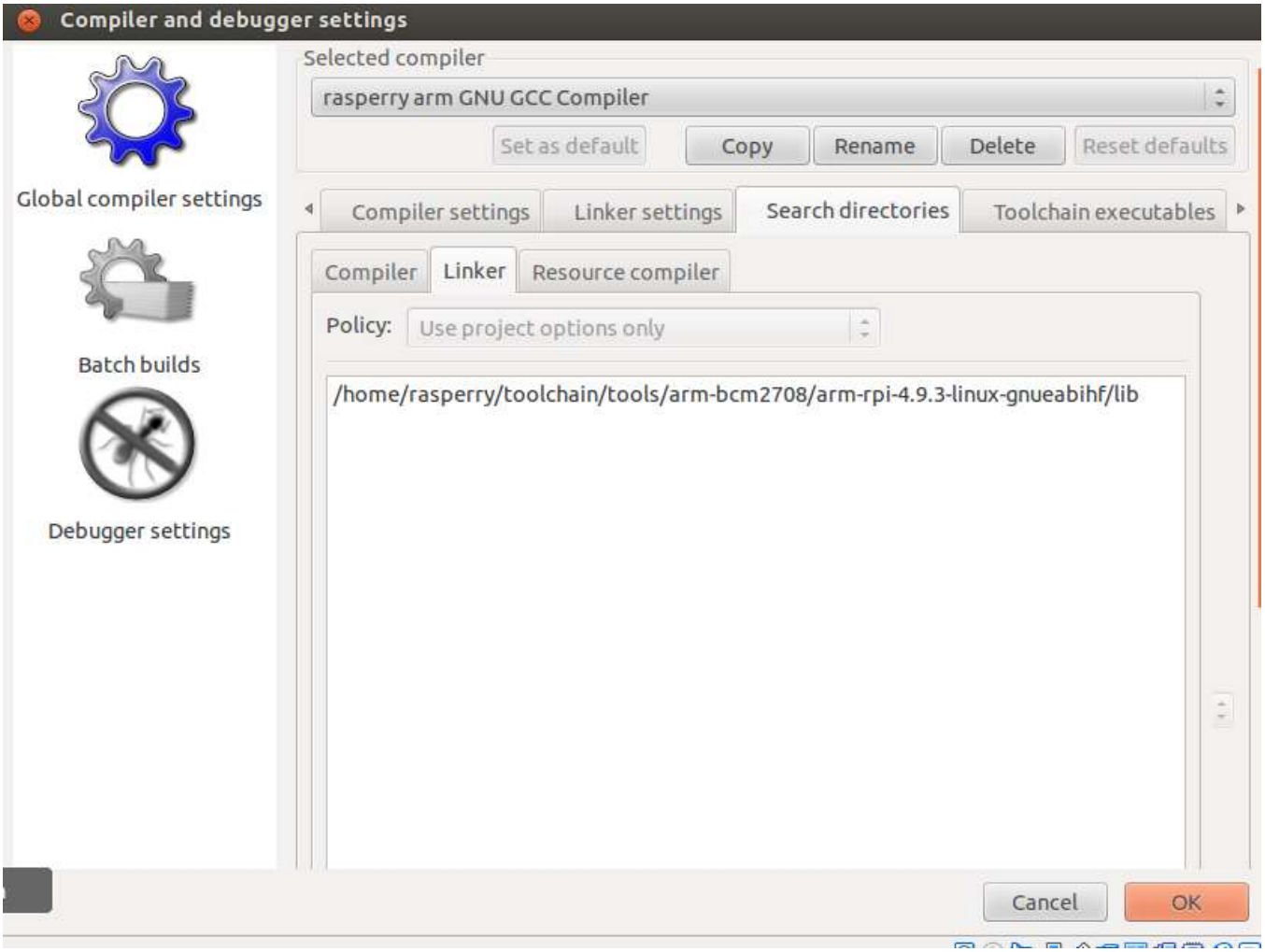
Compiler settings | Linker settings | Search directories | Toolchain executables

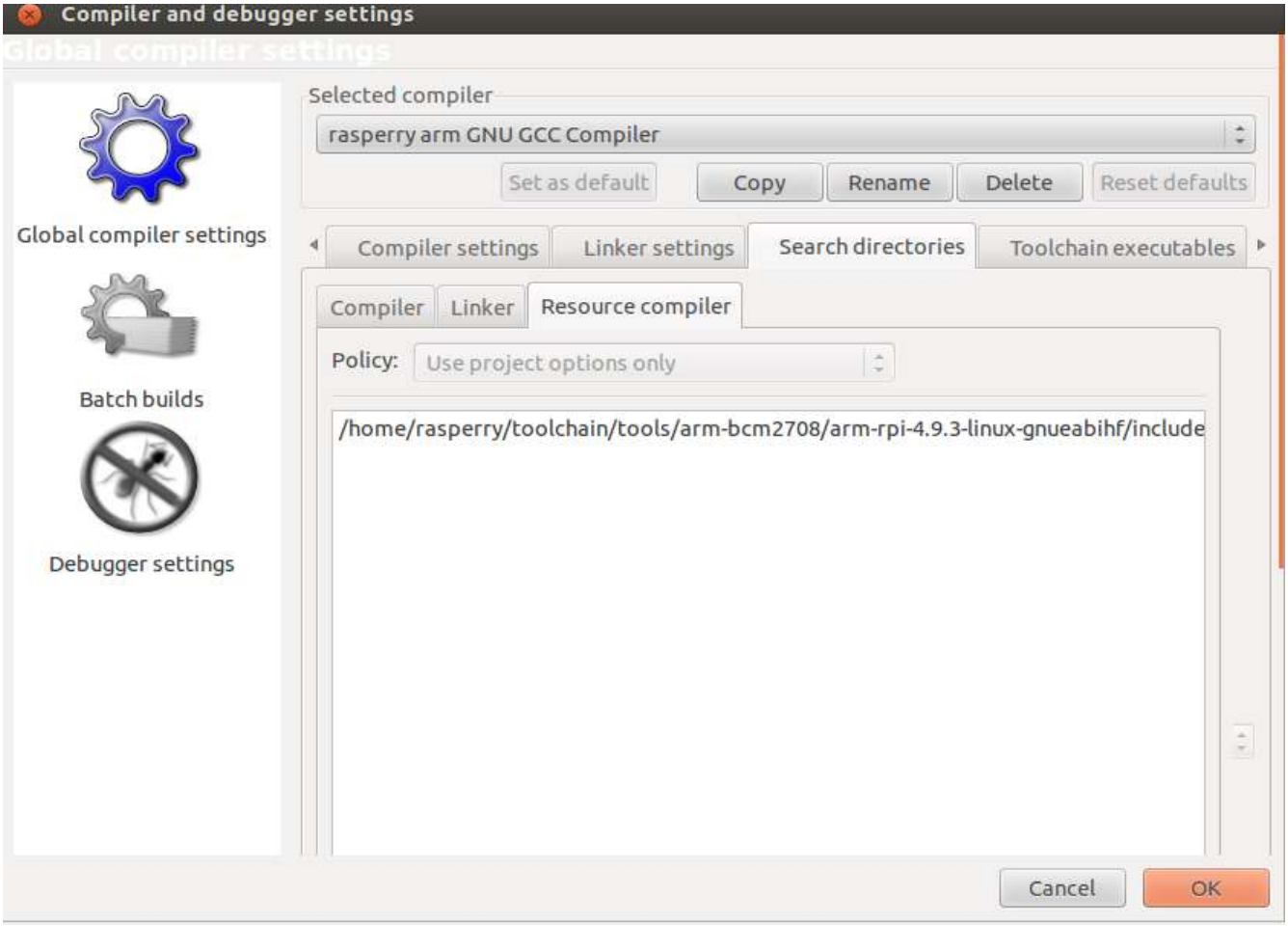
Compiler | Linker | Resource compiler

Policy: Use project options only

/home/rasperry/toolchain/tools/arm-bcm2708/arm-rpi-4.9.3-linux-gnueabihf/include

Cancel | OK


Compiler and debugger settings
Global compiler settings
Batch builds
Debugger settings
Selected compiler
rasperry arm GNU GCC Compiler
Set as default
Copy
Rename
Delete
Reset defaults
Compiler settings
Linker settings
Search directories
Toolchain executables
Compiler
Linker
Resource compiler
Policy:
Use project options only
/home/rasperry/toolchain/tools/arm-bcm2708/arm-rpi-4.9.3-linux-gnueabihf/lib
Cancel
OK


Compiler and debugger settings
Global compiler settings
Batch builds
Debugger settings
Selected compiler
rasperry arm GNU GCC Compiler
Set as default
Copy
Rename
Delete
Reset defaults
Compiler settings
Linker settings
Search directories
Toolchain executables
Compiler
Linker
Resource compiler
Policy:
Use project options only
/home/rasperry/toolchain/tools/arm-bcm2708/arm-rpi-4.9.3-linux-gnueabihf/include
Cancel
OK

Beagle bone Serial port setup for IEC 60870-5-101 and DNP3 Communication

Task: Display Detected System's Serial Support

dmesg | **grep** tty

For my quadcopter project, the BBB must enable all UART ports by itself at boot, since I can't fly with a wire hooked onto it. Therefore, I went on and do some digging on this… took a while before anything came up. It's actually more complicated than I though, having to deal with how device tree works and all… but here's what I found:

The Easy Method (Only Work for UART1, UART2, UART4, UART5)
First, we want to check out what device trees are available for us to use…

```
// Navigation to the Device Tree File Directory
cd /lib/firmware/
ls
```

Okay, there's lots of files there, but what really matter to us are the UART files:

```
BB-UART1-00A0.dtbo
BB-UART2-00A0.dtbo
BB-UART4-00A0.dtbo
BB-UART5-00A0.dtbo
```

As we can see, UART3, like always is not available… that's because its RX pin is tied to HDMI. UART5 is also conflicting with the LCD cape, or so I heard from elsewhere…

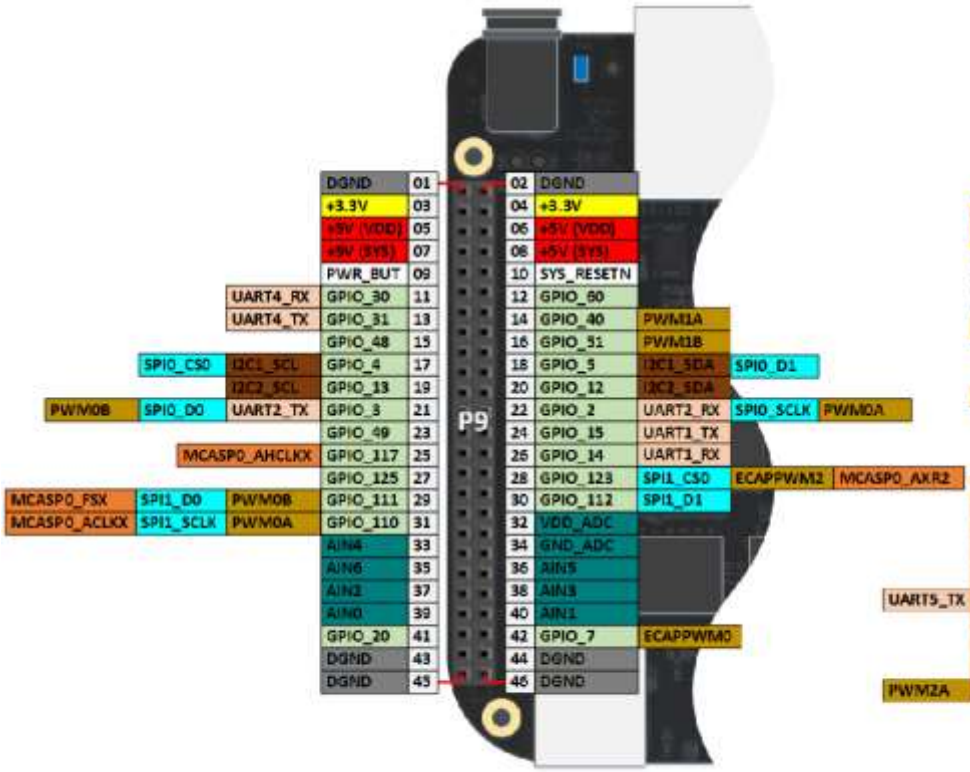
Anyway, these are the device tree files that we actually used to activate manually. So now, we will need to edit the boot file **uEnv.txt** to automatically activate those files. Note, armhf's files, unfortunately don't work here, so we can't use this method to enable UART3. We could have use Adafruit's library to write a python script to setup the UART and launch it at boot with Cron. But Adafruit don't support UART3 and UART4… that's why we have the longer method later on…

```
\\ Change to Boot Directory and Edit uEnv.txt (note, we are using Debian here...)
cd /boot/uboot/
sudo nano uEnv.txt
```

Add the following line after all the cape stuffs:

```
optargs=capemgr.enable_partno=BB-UART1,BB-UART2,BB-UART3,BB-UART5
```

Save the file with **Ctrl+X**, **Y**, **Enter**, and then **sudo reboot**, and we should be set.


P9
DGND 01
+3.3V 03
+5V (VDD) 05
+5V (SYS) 07
PWR_BUT 09
UART4_RX GPIO_30 11
UART4_TX GPIO_31 13
GPIO_48 15
SPI0_CS0 I2C1_SCL GPIO_4 17
I2C2_SCL GPIO_13 19
PWM0B SPI0_D0 UART2_TX GPIO_3 21
GPIO_49 23
MCASP0_AHCLKX GPIO_117 25
GPIO_125 27
MCASP0_FSX SPI1_D0 PWM0B GPIO_111 29
MCASP0_ACLKX SPI1_SCLK PWM0A GPIO_110 31
AIN4 33
AIN6 35
AIN2 37
AIN0 39
GPIO_20 41
DGND 43
DGND 45
02 DGND
04 +3.3V
06 +5V (VDD)
08 +5V (SYS)
10 SYS_RESETN
12 GPIO_60
14 GPIO_40 PWM1A
16 GPIO_51 PWM1B
18 GPIO_5 I2C1_SDA SPI0_D1
20 GPIO_12 I2C2_SDA
22 GPIO_2 UART2_RX SPI0_SCLK PWM0A
24 GPIO_15 UART1_TX
26 GPIO_14 UART1_RX
28 GPIO_123 SPI1_CS0 ECAPPWM2 MCASP0_AXR2
30 GPIO_112 SPI1_D1
32 VDD_ADC
34 GND_ADC
36 AIN5
38 AIN3
40 AIN1
42 GPIO_7 ECAPPWM0
44 DGND
46 DGND
P8
DGND 01
MMC1_DAT6 GPIO_38 03
MMC1_DAT2 GPIO_34 05
TIMER4 GPIO_66 07
TIMER2 GPIO_69 09
GPIO_45 11
PWM2B GPIO_23 13
GPIO_47 15
GPIO_27 17
PWM2A GPIO_22 19
GPIO_62 21
GPIO_36 23
GPIO_32 25
LCD_VSYNC GPIO_86 27
LCD_HSYNC GPIO_87 29
LCD_DATA14 GPIO_10 31
LCD_DATA13 GPIO_9 33
LCD_DATA12 GPIO_8 35
UART5_TX LCD_DATA8 GPIO_78 37
LCD_DATA6 GPIO_76 39
LCD_DATA4 GPIO_74 41
LCD_DATA2 GPIO_72 43
PWM2A LCD_DATA0 GPIO_70 45
02 DGND
04 GPIO_39 MMC1_DAT7
06 GPIO_35 MMC1_DAT3
08 GPIO_67 TIMER7
10 GPIO_68 TIMER1
12 GPIO_44
14 GPIO_26
16 GPIO_46
18 GPIO_65
20 GPIO_63 MMC1_CMD
22 GPIO_37 MMC1_DAT5
24 GPIO_33 MMC1_DAT1
26 GPIO_61
28 GPIO_88 LCD_PCLK
30 GPIO_89 LCD_AC_BIAS
32 GPIO_11 LCD_DATA15
34 GPIO_81 LCD_DATA11
36 GPIO_80 LCD_DATA10
38 GPIO_79 LCD_DATA19
40 GPIO_77 LCD_DATA7
42 GPIO_75 LCD_DATA5
44 GPIO_73 LCD_DATA3
46 GPIO_71 LCD_DATA1